Android, MySQL, PHP, & JSON 5 | Login & Reg Android App Tutorial

Posted By trav on Jun 10, 2013 | 0 comments

This entry is part 21 of 22 in the series Android-Intermediate

Creating the Android Application to Interact with our Remote Database!

In this Android Remote Databases mini series we'll create our Android application to parse the JSON data that our PHP scripts output. Our Android application will be able post data to our PHP scripts to register a new user, sign in, add a comment, or display all of the current comments.

Follow the Remote Databases for Android Series!

- Part 1: Android Remote Databases and Servers Overview
- Part 2: Local Xampp Server and MySQL Database
- Part 3: Working with a Remote Server and MySQL
- Part 4: Creating Login and Registration PHP Scripts
- **Part 5: Developing the Android Application**
- Part 6: JSON Parsing and Android ListView Design

Remote

Databases Part 5 Video Tutorial!

Coming soon..

Setting up our Android Project

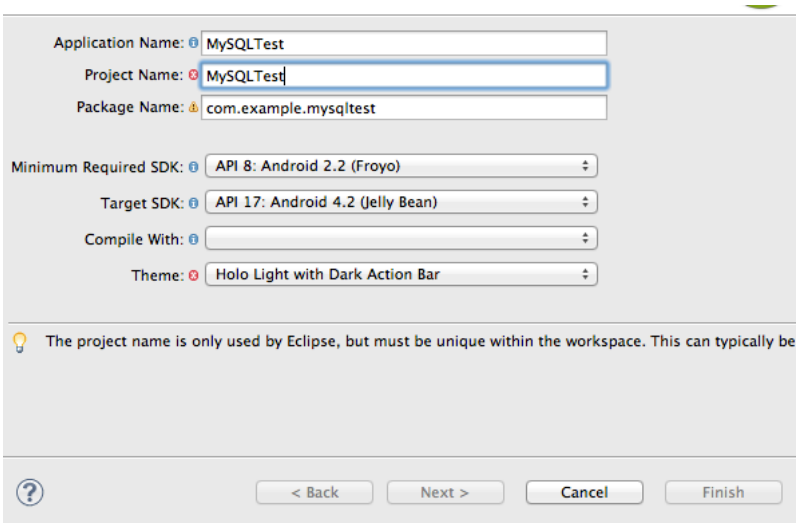
The time has finally come! We have everything setup for us to be able to interact with our MySQL remote database from within an Android application. So, let's create it! In this tutorial we will develop the login and register features of our app.

**You should be familiar with the basics of setting up an Android projects and the structure of an Android project. If at any time you are confused, you can check out our Android Basics Course.

Step 1: Creating a New Android Project

Even though Android Studios just came out, I'm going to use Eclipse to setup my new Android Project. I gave it the following credentials:

- Application Name: MySQLTest
- Project Name: MySQLTest
- Package Name: com.example.mysqltest



Step 2: Setting Up Classes and Layouts

Create the following Java classes and xml layouts:

Java files

- Login.java
- Register.java
- AddComment.java
- ReadComments.java
- JSONParser.java

Android XML Layouts

- login.xml
- register.xml
- add_comment.xml
- read_comments.xml
- single_comment.xml

Step 3: Set up the Permissions and the Manifest

In order for use to interact with our PHP scripts and our remote MySQL database, we will need to make sure our Android application has the internet permission. Since we are modifying our manifest, we might as well add all of the activities we are going to have in our application as well.

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.mysqltest"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="17" />
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name="com.example.mysqltest.Login"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name="com.example.mysqltest.Register"
            android:label="@string/app_name"  >
        </activity>
        <activity
            android:name="com.example.mysqltest.AddComment"
            android:label="@string/app_name"  >
        </activity>
         <activity
            android:name="com.example.mysqltest.ReadComments"
            android:label="@string/app_name"  >
        </activity>
    </application>
</manifest>
```

Step 4: Creating Simple Login and Register Layouts

I'm not the best when it comes to designing xml layouts. Especially, when I'm creating tutorials, so please bare with me here, these are going to be some ugly layouts for now.

Here is the image I'm using. Right-click on this bad boy, choose "Save as..", and save the image within your Android project's drawable folder as "arrowstars.png". (I have no idea why I have our logo labeled as arrowstars, but meh, it works!)

login.xml

```xml
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
   >

    <Button
        android:id="@+id/register"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/login"
        android:layout_alignParentBottom="true"
        android:layout_alignRight="@+id/login"
        android:layout_marginBottom="25dp"
        android:text="Register" />

```

```
    <Button
        android:id="@+id/login"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"

        android:layout_above="@+id/register"
        android:layout_alignLeft="@+id/password"
        android:layout_alignRight="@+id/password"
        android:text="Login" />

    <EditText
        android:id="@+id/password"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/login"
        android:layout_centerHorizontal="true"
        android:ems="10"
        android:inputType="textPassword" >

        <requestFocus />
    </EditText>

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="17dp"
        android:gravity="center"
        android:text="Android Remote Server Tutorial"
        android:textAppearance="?android:attr/textAppearanceLarge"
        android:textStyle="bold" />

    <ImageView
        android:id="@+id/imageView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView2"
        android:layout_centerHorizontal="true"
        android:src="@drawable/arrowstars" />

    <TextView
        android:id="@+id/TextView01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/password"
        android:layout_alignLeft="@+id/password"
        android:layout_marginLeft="22dp"
        android:text="Password" />

    <EditText
        android:id="@+id/username"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/TextView01"
        android:layout_centerHorizontal="true"
        android:ems="10" />

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignRight="@+id/TextView01"
        android:layout_centerVertical="true"
        android:text="Username" />

</RelativeLayout>
```

register.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
   >

    <ImageView
        android:id="@+id/imageView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView2"
        android:layout_centerHorizontal="true"
        android:src="@drawable/arrowstars" />

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/password"
        android:layout_centerVertical="true"
```

```
        android:text="Username" />

    <EditText
        android:id="@+id/username"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textView1"
        android:ems="10" />

    <TextView
        android:id="@+id/TextView01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/username"
        android:layout_below="@+id/username"
        android:text="Password" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true"
        android:layout_marginTop="16dp"
        android:gravity="center"
        android:text="Android Remote Server Tutorial"
        android:textAppearance="?android:attr/textAppearanceLarge"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/password"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/TextView01"
        android:layout_centerHorizontal="true"
        android:ems="10"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/register"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignRight="@+id/password"
        android:layout_below="@+id/password"
        android:text="Register" />

</RelativeLayout>
```

I know I'm not using the standard string references for my text values, but don't worry about that, we can fix everything later.

Step 5: Creating Our Login and Register Activities.

Now, the fun begins, we need to test our login system. First, let create a simple success page for our ReadComments.java and read_comments.xml

read_comments.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#fff" >

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Success!"
        android:textAppearance="?android:attr/textAppearanceLarge" />

</LinearLayout>
```

ReadComments.java

```
package com.example.mysqltest;

import android.app.Activity;
import android.os.Bundle;

public class ReadComments extends Activity{
```

```
7
8       @Override
9      protected void onCreate(Bundle savedInstanceState) {
10         // TODO Auto-generated method stub
11         super.onCreate(savedInstanceState);
12         setContentView(R.layout.read_comments);
13     }
14 }
```

That was simple enough! Now, we need to create a class that will help us parse some JSON information. There are a lot of JSON parsing classes for Android on the internet you can use, or you could create your own. To save time, I'm going to use one I found on the internet, and modify it to our needs.

```
1  package com.example.mysqltest;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStream;
6  import java.io.InputStreamReader;
7  import java.io.UnsupportedEncodingException;
8
9  import org.apache.http.HttpEntity;
10 import org.apache.http.HttpResponse;
11 import org.apache.http.client.ClientProtocolException;
12 import org.apache.http.client.methods.HttpPost;
13 import org.apache.http.impl.client.DefaultHttpClient;
14 import org.json.JSONException;
15 import org.json.JSONObject;
16
17 import android.util.Log;
18
19 public class JSONParser {
20
21     static InputStream is = null;
22     static JSONObject jObj = null;
23     static String json = "";
24
25     // constructor
26     public JSONParser() {
27
28     }
29
30     public JSONObject getJSONFromUrl(String url) {
31
32         // Making HTTP request
33         try {
34             // defaultHttpClient
35             DefaultHttpClient httpClient = new DefaultHttpClient();
36             HttpPost httpPost = new HttpPost(url);
37
38             HttpResponse httpResponse = httpClient.execute(httpPost);
39             HttpEntity httpEntity = httpResponse.getEntity();
40             is = httpEntity.getContent();
41
42         } catch (UnsupportedEncodingException e) {
43             e.printStackTrace();
44         } catch (ClientProtocolException e) {
45             e.printStackTrace();
46         } catch (IOException e) {
47             e.printStackTrace();
48         }
49
50         try {
51             BufferedReader reader = new BufferedReader(new InputStreamReader(
52                     is, "iso-8859-1"), 8);
53             StringBuilder sb = new StringBuilder();
54             String line = null;
55             while ((line = reader.readLine()) != null) {
56                 sb.append(line + "\n");
57             }
58             is.close();
59             json = sb.toString();
60         } catch (Exception e) {
61             Log.e("Buffer Error", "Error converting result " + e.toString());
62         }
63
64         // try parse the string to a JSON object
65         try {
66             jObj = new JSONObject(json);
67         } catch (JSONException e) {
68             Log.e("JSON Parser", "Error parsing data " + e.toString());
69         }
70
71         // return JSON String
72         return jObj;
73
74     }
75 }
```

This JSONParser.java class may look like a bunch of baloney to you right now, but this is the class that allows us to interpret the JSON data our php web service spits out. With this class we will be able to read whether or not a login was successful (with either a 0 or 1). We will also be able to list out all of our posts in an orderly fashion using this JSON parser. We will go over the details later. As for now lets setup our Login Activity:

Login.java v0.1 – Basics

```
package com.example.mysqltest;

import android.app.Activity;
import android.app.ProgressDialog;
import android.content.Intent;
import android.os.AsyncTask;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;

public class Login extends Activity implements OnClickListener{

    private EditText user, pass;
    private Button mSubmit, mRegister;

     // Progress Dialog
    private ProgressDialog pDialog;

    // JSON parser class
    JSONParser jsonParser = new JSONParser();

    //php login script location:

    //localhost :
    //testing on your device
    //put your local ip instead,  on windows, run CMD > ipconfig
    //or in mac's terminal type ifconfig and look for the ip under en0 or en1
   // private static final String LOGIN_URL =
"http://xxx.xxx.x.x:1234/webservice/login.php";

    //testing on Emulator:
    private static final String LOGIN_URL =
"http://10.0.2.2:1234/webservice/login.php";

  //testing from a real server:
    //private static final String LOGIN_URL =
"http://www.yourdomain.com/webservice/login.php";

    //JSON element ids from repsonse of php script:
    private static final String TAG_SUCCESS = "success";
    private static final String TAG_MESSAGE = "message";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // TODO Auto-generated method stub
        super.onCreate(savedInstanceState);
        setContentView(R.layout.login);

        //setup input fields
        user = (EditText)findViewById(R.id.username);
        pass = (EditText)findViewById(R.id.password);

        //setup buttons
        mSubmit = (Button)findViewById(R.id.login);
        mRegister = (Button)findViewById(R.id.register);

        //register listeners
        mSubmit.setOnClickListener(this);
        mRegister.setOnClickListener(this);

    }

    @Override

    public void onClick(View v) {
        // determine which button was pressed:
        switch (v.getId()) {
        case R.id.login:
                new AttemptLogin().execute();
            break;
        case R.id.register:
                Intent i = new Intent(this, Register.class);
                startActivity(i);
            break;

        default:
            break;
        }
    }

```

```
    //AsyncTask is a seperate thread than the thread that runs the GUI
    //Any type of networking should be done with asynctask.
    class AttemptLogin extends AsyncTask<String, String, String> {

        //three methods get called, first preExecture, then do in background, and
once do
        //in back ground is completed, the onPost exection method will be called.

        @Override
        protected void onPreExecute() {
            super.onPreExecute();

        }

        @Override
        protected String doInBackground(String... args) {

            return null;

        }

        protected void onPostExecute(String file_url) {

        }

    }

}
```

This java file looks pretty basic so far. The important part is that we are using AsyncTask because connecting our php scripts may take some time, and therefore we don't want to use the same thread that runs the GUI of our application. If we did, it would really slow things down, freeze usability for our user, and maybe even crash our application. AsyncTask will create another thread to execute the tasks we want completed.

You also want to note where your PHP scripts are located. If you are using Xampp as a local server, make sure you uncomment the appropriate LOGIN_URL. If your scripts are on a shared server somewhere, use your domain name as the LOGIN_URL.

Here is the break down of what we want our login java class to do:

- If the register button is clicked, open the register activity.
- If the login button is clicked, create a dialog that pops up before trying to connect to the scripts.
- Once the dialog is displayed, get the inputs from our "username" and "password" EditTexts.
- POST that data to our URL that handles logging in.
- Retrieve the JSON response from our server.
- Interpret the response.
- If the response element "success" is 1, finish the Login Activity, and open the ReadComments Activity.
- If the response element "success" is 0, do nothing.
- In either case, we will close our dialog.
- Lastly, we will display the JSON response for the element "message" as a toast.

Awesome, let's make it happen.

Login.java v0.1 – Basics

```
package com.example.mysqltest;

import java.util.ArrayList;
import java.util.List;

import org.apache.http.NameValuePair;
import org.apache.http.message.BasicNameValuePair;
import org.json.JSONException;
import org.json.JSONObject;

import android.app.Activity;
import android.app.ProgressDialog;
import android.content.Intent;
import android.os.AsyncTask;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

public class Login extends Activity implements OnClickListener{

    private EditText user, pass;
    private Button mSubmit, mRegister;

```

```
28      // Progress Dialog
29      private ProgressDialog pDialog;
30
31      // JSON parser class
32      JSONParser jsonParser = new JSONParser();
33
34      //php login script location:
35
36      //localhost :
37      //testing on your device
38      //put your local ip instead,  on windows, run CMD > ipconfig
39      //or in mac's terminal type ifconfig and look for the ip under en0 or en1
40     // private static final String LOGIN_URL =
"http://xxx.xxx.x.x:1234/webservice/login.php";
41
42      //testing on Emulator:
43      private static final String LOGIN_URL =
"http://10.0.2.2:1234/webservice/login.php";
44
45    //testing from a real server:
46      //private static final String LOGIN_URL =
"http://www.yourdomain.com/webservice/login.php";
47
48      //JSON element ids from repsonse of php script:
49      private static final String TAG_SUCCESS = "success";
50      private static final String TAG_MESSAGE = "message";
51
52      @Override
53      protected void onCreate(Bundle savedInstanceState) {
54          // TODO Auto-generated method stub
55          super.onCreate(savedInstanceState);
56          setContentView(R.layout.login);
57
58          //setup input fields
59          user = (EditText)findViewById(R.id.username);
60          pass = (EditText)findViewById(R.id.password);
61
62          //setup buttons
63          mSubmit = (Button)findViewById(R.id.login);
64          mRegister = (Button)findViewById(R.id.register);
65
66          //register listeners
67          mSubmit.setOnClickListener(this);
68          mRegister.setOnClickListener(this);
69
70      }
71
72      @Override
73      public void onClick(View v) {
74          // TODO Auto-generated method stub
75          switch (v.getId()) {
76          case R.id.login:
77                  new AttemptLogin().execute();
78              break;
79          case R.id.register:
80                  Intent i = new Intent(this, Register.class);
81                  startActivity(i);
82              break;
83
84          default:
85              break;
86          }
87      }
88
89      class AttemptLogin extends AsyncTask<String, String, String> {
90
91           /**
92           * Before starting background thread Show Progress Dialog
93           * */
94          boolean failure = false;
95
96          @Override
97          protected void onPreExecute() {
98              super.onPreExecute();
99              pDialog = new ProgressDialog(Login.this);
100             pDialog.setMessage("Attempting login...");
101             pDialog.setIndeterminate(false);
102             pDialog.setCancelable(true);
103             pDialog.show();
104         }
105
106         @Override
107         protected String doInBackground(String... args) {
108             // TODO Auto-generated method stub
109              // Check for success tag
110             int success;
111             String username = user.getText().toString();
112             String password = pass.getText().toString();
113             try {
114                 // Building Parameters
```

```
                List<NameValuePair> params = new ArrayList<NameValuePair>();
                params.add(new BasicNameValuePair("username", username));
                params.add(new BasicNameValuePair("password", password));

                Log.d("request!", "starting");
                // getting product details by making HTTP request
                JSONObject json = jsonParser.makeHttpRequest(
                       LOGIN_URL, "POST", params);

                // check your log for json response
                Log.d("Login attempt", json.toString());

                // json success tag
                success = json.getInt(TAG_SUCCESS);
                if (success == 1) {
                    Log.d("Login Successful!", json.toString());
                    Intent i = new Intent(Login.this, ReadComments.class);
                    finish();
                    startActivity(i);
                    return json.getString(TAG_MESSAGE);
                }else{
                    Log.d("Login Failure!", json.getString(TAG_MESSAGE));
                    return json.getString(TAG_MESSAGE);

                }
            } catch (JSONException e) {
                e.printStackTrace();
            }

            return null;

        }
        /**
         * After completing background task Dismiss the progress dialog
         * **/
        protected void onPostExecute(String file_url) {
            // dismiss the dialog once product deleted
            pDialog.dismiss();
            if (file_url != null){
                Toast.makeText(Login.this, file_url, Toast.LENGTH_LONG).show();
            }

        }

    }
}
```

Reading JSON data is as simple as that! We simple POST the information to the url we want within the make HTTP request, and we parse the result! You can now look at the JSONParser.java class and see that it is quite a simple class. All that it does is determine what type of method to use, either POST or GET, and then it makes a httprequest and gets the response. Lastly, the JSONParser class will use a buffer reader to get the different elements of the JSON object.

You also may have noticed that the we are calling a method from JSONParser class called makeHttpRequest. This is a method that will read the JSON data an put everything in the appropriate list. Let's add modify our JSONParser class to include this method.

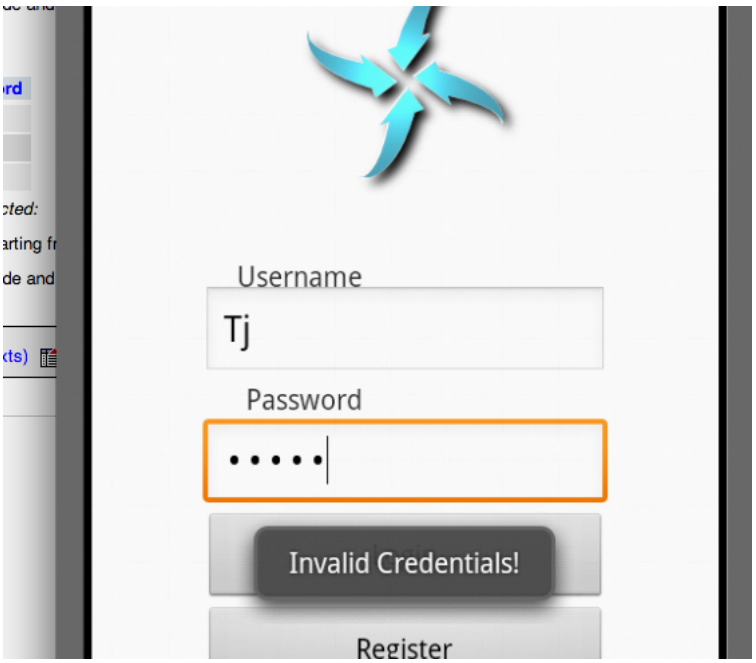
JSONParser.java

```
package com.example.mysqltest;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.util.List;

import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.NameValuePair;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.entity.UrlEncodedFormEntity;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.client.utils.URLEncodedUtils;
import org.apache.http.impl.client.DefaultHttpClient;
import org.json.JSONException;
import org.json.JSONObject;

import android.util.Log;

public class JSONParser {

    static InputStream is = null;
```

```
27      static JSONObject jObj = null;
28      static String json = "";
29
30      // constructor
31      public JSONParser() {
32
33      }
34
35
36      public JSONObject getJSONFromUrl(final String url) {
37
38          // Making HTTP request
39          try {
40              // Construct the client and the HTTP request.
41              DefaultHttpClient httpClient = new DefaultHttpClient();
42              HttpPost httpPost = new HttpPost(url);
43
44              // Execute the POST request and store the response locally.
45              HttpResponse httpResponse = httpClient.execute(httpPost);
46              // Extract data from the response.
47              HttpEntity httpEntity = httpResponse.getEntity();
48              // Open an inputStream with the data content.
49              is = httpEntity.getContent();
50
51          } catch (UnsupportedEncodingException e) {
52              e.printStackTrace();
53          } catch (ClientProtocolException e) {
54              e.printStackTrace();
55          } catch (IOException e) {
56              e.printStackTrace();
57          }
58
59          try {
60              // Create a BufferedReader to parse through the inputStream.
61              BufferedReader reader = new BufferedReader(new InputStreamReader(
62                      is, "iso-8859-1"), 8);
63              // Declare a string builder to help with the parsing.
64              StringBuilder sb = new StringBuilder();
65              // Declare a string to store the JSON object data in string form.
66              String line = null;
67
68              // Build the string until null.
69              while ((line = reader.readLine()) != null) {
70                  sb.append(line + "\n");
71              }
72
73              // Close the input stream.
74              is.close();
75              // Convert the string builder data to an actual string.
76              json = sb.toString();
77          } catch (Exception e) {
78              Log.e("Buffer Error", "Error converting result " + e.toString());
79          }
80
81          // Try to parse the string to a JSON object
82          try {
83              jObj = new JSONObject(json);
84          } catch (JSONException e) {
85              Log.e("JSON Parser", "Error parsing data " + e.toString());
86          }
87
88          // Return the JSON Object.
89          return jObj;
90
91      }
92
93
94      // function get json from url
95      // by making HTTP POST or GET mehtod
96      public JSONObject makeHttpRequest(String url, String method,
97              List<NameValuePair> params) {
98
99          // Making HTTP request
100         try {
101
102             // check for request method
103             if(method == "POST"){
104                 // request method is POST
105                 // defaultHttpClient
106                 DefaultHttpClient httpClient = new DefaultHttpClient();
107                 HttpPost httpPost = new HttpPost(url);
108                 httpPost.setEntity(new UrlEncodedFormEntity(params));
109
110                 HttpResponse httpResponse = httpClient.execute(httpPost);
111                 HttpEntity httpEntity = httpResponse.getEntity();
112                 is = httpEntity.getContent();
113
114             }else if(method == "GET"){
115                 // request method is GET
116                 DefaultHttpClient httpClient = new DefaultHttpClient();
117                 String paramString = URLEncodedUtils.format(params, "utf-8");
```

```
                url += "?" + paramString;
                HttpGet httpGet = new HttpGet(url);

                HttpResponse httpResponse = httpClient.execute(httpGet);
                HttpEntity httpEntity = httpResponse.getEntity();
                is = httpEntity.getContent();
            }

        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        } catch (ClientProtocolException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        try {
            BufferedReader reader = new BufferedReader(new InputStreamReader(
                    is, "iso-8859-1"), 8);
            StringBuilder sb = new StringBuilder();
            String line = null;
            while ((line = reader.readLine()) != null) {
                sb.append(line + "\n");
            }
            is.close();
            json = sb.toString();
        } catch (Exception e) {
            Log.e("Buffer Error", "Error converting result " + e.toString());
        }

        // try parse the string to a JSON object
        try {
            jObj = new JSONObject(json);
        } catch (JSONException e) {
            Log.e("JSON Parser", "Error parsing data " + e.toString());
        }

        // return JSON String
        return jObj;

    }
}
```

Another thing I wanted to mention is that, we don't want to create a toast within the "doInBackground" method of our AsyncTask class, instead, we want to pass the response to the onPostExecute method and display the toast there! Again, this toast is displaying whatever the "message" JSON element contains, and therefore, as a front-end Android developer, you wouldn't have to worry about what it says since that would be the back-end programmer's job. As you can see here, this is what is displayed when you try to login with a unregistered username:



Now, let's outline what we want the **Register.java** class should do:

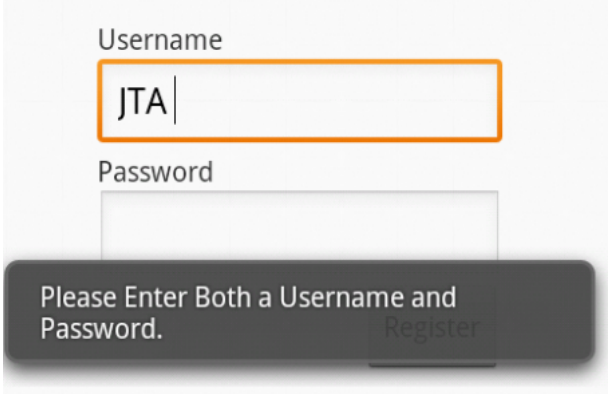
- Try to register user.
- If JSON element "success" is 1, display success message and close registration activity.
- If JSON element "success" is 0, display message explaining why registration is unsuccessful.
- That's it.

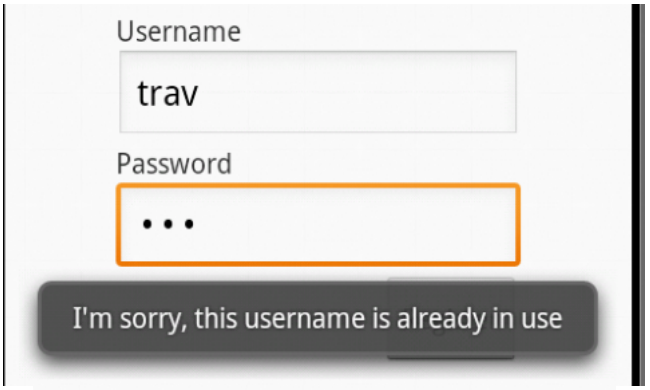
Register.java

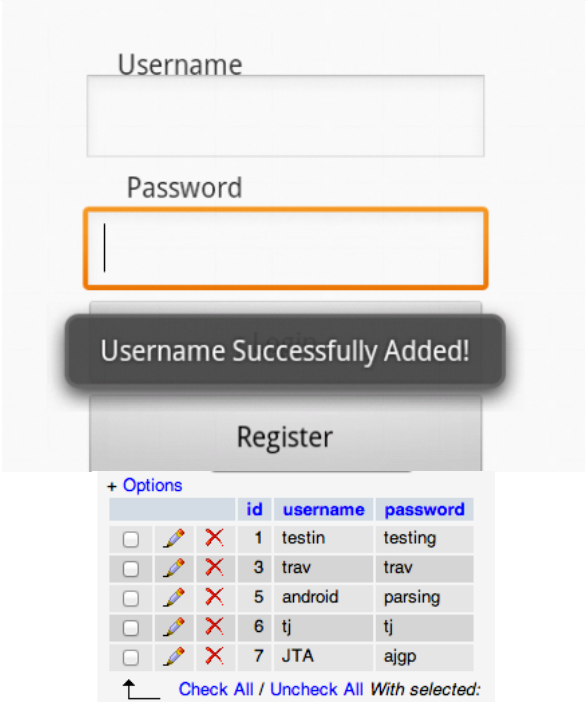
```
package com.example.mysqltest;

import java.util.ArrayList;
import java.util.List;
import org.apache.http.NameValuePair;
import org.apache.http.message.BasicNameValuePair;
import org.json.JSONException;
import org.json.JSONObject;
import android.app.Activity;
import android.app.ProgressDialog;
import android.os.AsyncTask;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

public class Register extends Activity implements OnClickListener{

    private EditText user, pass;
    private Button  mRegister;

     // Progress Dialog
    private ProgressDialog pDialog;

    // JSON parser class
    JSONParser jsonParser = new JSONParser();

    //php login script

    //localhost :
    //testing on your device
    //put your local ip instead,  on windows, run CMD > ipconfig
    //or in mac's terminal type ifconfig and look for the ip under en0 or en1
   // private static final String LOGIN_URL =
"http://xxx.xxx.x.x:1234/webservice/register.php";

    //testing on Emulator:
    private static final String LOGIN_URL =
"http://10.0.2.2:1234/webservice/register.php";

  //testing from a real server:
    //private static final String LOGIN_URL =
"http://www.yourdomain.com/webservice/register.php";

    //ids
    private static final String TAG_SUCCESS = "success";
    private static final String TAG_MESSAGE = "message";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // TODO Auto-generated method stub
        super.onCreate(savedInstanceState);
        setContentView(R.layout.register);

        user = (EditText)findViewById(R.id.username);
        pass = (EditText)findViewById(R.id.password);

        mRegister = (Button)findViewById(R.id.register);
        mRegister.setOnClickListener(this);

    }

    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub

                new CreateUser().execute();

    }

    class CreateUser extends AsyncTask<String, String, String> {

         /**
         * Before starting background thread Show Progress Dialog
         * */
        boolean failure = false;

        @Override
        protected void onPreExecute() {
            super.onPreExecute();
            pDialog = new ProgressDialog(Register.this);
            pDialog.setMessage("Creating User...");
            pDialog.setIndeterminate(false);
            pDialog.setCancelable(true);
            pDialog.show();
```

```
        }

        @Override
        protected String doInBackground(String... args) {
            // TODO Auto-generated method stub
             // Check for success tag
            int success;
            String username = user.getText().toString();
            String password = pass.getText().toString();
            try {
                // Building Parameters
                List<NameValuePair> params = new ArrayList<NameValuePair>();
                params.add(new BasicNameValuePair("username", username));
                params.add(new BasicNameValuePair("password", password));

                Log.d("request!", "starting");

                //Posting user data to script
                JSONObject json = jsonParser.makeHttpRequest(
                       LOGIN_URL, "POST", params);

                // full json response
                Log.d("Login attempt", json.toString());

                // json success element
                success = json.getInt(TAG_SUCCESS);
                if (success == 1) {
                    Log.d("User Created!", json.toString());
                    finish();
                    return json.getString(TAG_MESSAGE);
                }else{
                    Log.d("Login Failure!", json.getString(TAG_MESSAGE));
                    return json.getString(TAG_MESSAGE);

                }
            } catch (JSONException e) {
                e.printStackTrace();
            }

            return null;

        }
        /**
         * After completing background task Dismiss the progress dialog
         * **/
        protected void onPostExecute(String file_url) {
            // dismiss the dialog once product deleted
            pDialog.dismiss();
            if (file_url != null){
                Toast.makeText(Register.this, file_url, Toast.LENGTH_LONG).show();
            }

        }

    }

}
```

{"success":0,"message":"Please Enter Both a Username and Password."}







The registration activity looks pretty similar to the login activity. All we do, is try to register a user by POSTing data to our registration script, if it is successful, cool. If not, display a message explaining why.

Download the Source!

Download Source

That's it for this tutorial!

Support the Cause!

We have successfully communicated with our remote server and MySQL database! In the next tutorial we will create our other two activities that will post comments and parse comments. We will also work a little bit with designing a listview to look a bit more stylish (if we have time). Hopefully, you are starting to realize how easy it really is to interact with remote servers and to parse JSON data. I look forward to seeing you in the next tutorial!

Previous Lesson Donate Next Lesson

Author: trav

I'm just an average guy that love programming.

Share This Post On

Submit a Comment

Your email address will not be published. Required fields are marked *

Name *

Email *

Website

CAPTCHA Image

CAPTCHA Code *

Comment

You may use these HTML tags and attributes: `<a href="" title=""> <abbr title=""> <acronym title=""> <b> <blockquote cite=""> <cite> <code> <del datetime=""> <em> <i> <q cite=""> <strike> <strong>`

Submit Comment

Tutorials